

Track B VideoMAE Training on Colab - Complete Guide

This guide explains how to write a Colab notebook from scratch for Track B training with the ego-only VideoMAE backbone.

Data Required on Google Drive

1. VideoMAE Pretrained Checkpoint

Path: `Ego4d_STA/videomae/checkpoints/videomae_ego_scratch_last.pt`

What it is: HuggingFace VideoMAE encoder weights pretrained on egocentric clips.

Format:

```
{
  'model': {...},          # State dict with 'videomae.*' prefixed keys
  'optimizer': {...},     # Optimizer state
  'config': {              # HuggingFace VideoMAEConfig
    'hidden_size': 768,
    'num_frames': 16,
    'patch_size': 16,
    'tubelet_size': 2,
    'image_size': 224,
    ...
  },
  'epoch': 24              # Training epoch
}
```

Size: ~350 MB

2. Extracted Frames

Path: `ego4d_data/v2/extracted_frames/<video_uid>/<frame_number>.jpg`

What it is: JPEG frames extracted from Ego4D videos at specific timestamps.

Structure:

```
extracted_frames/
├── 002e11bc-deef-45f7-9af8-59421a606d69/
│   ├── 0000001.jpg
│   ├── 0000002.jpg
│   └── ...
```

```
| 00a8e40a-5b85-43e4-a7b3-4c2e4f1c8d3a/
|   ...
| ... (many UUIDs)
```

Frame specs:

- Resolution: Original (typically 540p or higher)
- Format: JPEG
- Naming: 7-digit zero-padded frame number

Size: ~10-50 GB (depends on number of clips)

3. Track A Stage B Manifests

Path: Ego4d_STA/runs/Track_A/trackA_stageB_YYYYMMDD_HHMMSS/

Files needed:

- head_train.jsonl - Training samples
- head_val.jsonl - Validation samples

JSONL format (one JSON object per line):

```
{
  "uid": "002e11bc-deef-45f7-9af8-59421a606d69",
  "frame_idx": 1234,
  "frame_name": "0001234.jpg",
  "candidates": [
    {
      "bbox": [x1, y1, x2, y2],
      "cls": 1,
      "ttc": 1.5,
      "ttc_norm": 0.3,
      "is_positive": 1
    },
    ...
  ]
}
```

Fields:

Field	Type	Description
uid	string	Video unique ID
frame_idx	int	Frame number
frame_name	string	Frame filename

Field	Type	Description
candidates	array	List of object candidates
candidates[].bbox	[x1,y1,x2,y2]	Bounding box coordinates
candidates[].cls	int	Class (1=next-active, 0=not)
candidates[].ttc	float	Time-to-contact in seconds
candidates[].ttc_norm	float	Normalized TTC (0-1)

Size: ~5-20 MB

Colab Notebook Structure

Cell 1: Check GPU

```
# Verify GPU is available
!nvidia-smi
import torch
print(f"CUDA available: {torch.cuda.is_available()}")
print(f"Device: {torch.cuda.get_device_name(0) if torch.cuda.is_available()
else 'CPU'}")
```

Cell 2: Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Cell 3: Install Dependencies

```
!pip install -q torch torchvision tqdm transformers pillow pyyaml
```

Cell 4: Clone Repository

```
import os
```

```
REPO_PATH = '/content/Ego4d-LiteSTA'

if not os.path.exists(REPO_PATH):
    !git clone https://github.com/saeedzns/Ego4d-LiteSTA.git {REPO_PATH}
else:
    !cd {REPO_PATH} && git pull
```

Cell 5: Configure Paths

```
import os
import sys

# === EDIT THESE PATHS TO MATCH YOUR DRIVE ===
DRIVE_ROOT = '/content/drive/MyDrive'

# VideoMAE checkpoint
VIDEOMAE_CKPT =
f'{DRIVE_ROOT}/Ego4d_STA/videomae/checkpoints/videomae_ego_scratch_last.pt'

# Extracted frames directory
FRAMES_ROOT = f'{DRIVE_ROOT}/ego4d_data/v2/extracted_frames'

# Track A manifests directory
MANIFESTS_ROOT = f'{DRIVE_ROOT}/Ego4d_STA/runs/Track_A'

# Verify paths
print("Checking paths...")
print(f"✓ VideoMAE checkpoint: {os.path.exists(VIDEOMAE_CKPT)}")
print(f"✓ Frames root: {os.path.exists(FRAMES_ROOT)}")
print(f"✓ Manifests root: {os.path.exists(MANIFESTS_ROOT)}")
```

Cell 6: Setup Local Directory Structure

```
import shutil
import glob

LOCAL_EXT = f'{REPO_PATH}/local_extraction'

# Create directories
os.makedirs(f'{LOCAL_EXT}/videomae_local_part/checkpoints_ego_scratch',
            exist_ok=True)
os.makedirs(f'{LOCAL_EXT}/v2', exist_ok=True)
os.makedirs(f'{LOCAL_EXT}/runs/Track_A', exist_ok=True)
```

```

# 1. Copy VideoMAE checkpoint
local_ckpt =
f'{LOCAL_EXT}/videomae_local_part/checkpoints_ego_scratch/videomae_ego_scratch
_last.pt'
if not os.path.exists(local_ckpt):
    print("Copying VideoMAE checkpoint...")
    shutil.copy(VIDEOMAE_CKPT, local_ckpt)
    print(f"✓ Copied to {local_ckpt}")

# 2. Symlink frames (don't copy - too large)
local_frames = f'{LOCAL_EXT}/v2/extracted_frames'
if not os.path.exists(local_frames):
    os.symlink(FRAMES_ROOT, local_frames)
    print(f"✓ Linked frames: {local_frames}")

# 3. Copy manifests (find latest stageB run)
stageb_runs = sorted(glob.glob(f'{MANIFESTS_ROOT}/trackA_stageB_*'))
if stageb_runs:
    latest_run = stageb_runs[-1]
    run_name = os.path.basename(latest_run)
    local_run = f'{LOCAL_EXT}/runs/Track_A/{run_name}'

    if not os.path.exists(local_run):
        shutil.copytree(latest_run, local_run)
        print(f"✓ Copied manifests: {local_run}")

    # Verify manifest files
    train_manifest = f'{local_run}/head_train.jsonl'
    val_manifest = f'{local_run}/head_val.jsonl'
    print(f"✓ Train manifest exists: {os.path.exists(train_manifest)}")
    print(f"✓ Val manifest exists: {os.path.exists(val_manifest)}")
else:
    print("✗ No Track A Stage B runs found!")

```

Cell 7: Set Working Directory

```

os.chdir(LOCAL_EXT)
sys.path.insert(0, LOCAL_EXT)
print(f"Working directory: {os.getcwd()}")

```

Cell 8: Test Config Loading

```

from core import load_config

```

```

cfg = load_config('trackB_videomae_ego', colab=True)

print("=== VideoMAE Config ===")
print(f"video_backbone: {cfg.get('model.tokenizer.video_backbone')}")
print(f"time_len: {cfg.get('model.tokenizer.time_len')}")
print(f"epochs: {cfg.get('training.epochs')}")
print(f"batch_size: {cfg.get('training.batch_size')}")
print(f"lr: {cfg.get('training.lr')}")
print(f"projector_in_dim: {cfg.get('model.projector.in_dim')}")

```

Cell 9: Test VideoMAE Checkpoint Loading

```

import torch

ckpt = torch.load(local_ckpt, map_location='cpu')
print(f"Checkpoint keys: {list(ckpt.keys())}")
print(f"Epoch: {ckpt.get('epoch', 'N/A')}")

if 'config' in ckpt:
    print(f"Hidden size: {ckpt['config'].get('hidden_size')}")
    print(f"Num frames: {ckpt['config'].get('num_frames')}")
    print(f"Patch size: {ckpt['config'].get('patch_size')}")

```

Cell 10: Run Demo (Quick Verification)

```

# Run 10-step demo to verify everything works
!python -m trackB.trackB_train_loader --config trackB_videomae_ego --demo

```

Expected output:

```

[TrackB] Loading config: trackB_videomae_ego
[TrackB] Video backbone: videomae_ego
[TrackB] Projector: 768 -> 256
[trackB.tokenizer] Loaded HuggingFace VideoMAE encoder...
[TrackB demo] steps (loader): 100%[██████████] 10/10 [00:XX<00:00]

```

Cell 11: Full Training

```
# Full training with VideoMAE backbone
# Adjust epochs as needed (20 epochs ~4-5 hours on T4 GPU)

!python -m trackB.trackB_train_loader \
    --config trackB_videomae_ego \
    --epochs 20 \
    --batch_size 8
```

Alternative: Quick test with fewer epochs:

```
!python -m trackB.trackB_train_loader \
    --config trackB_videomae_ego \
    --epochs 3 \
    --batch_size 8
```

Cell 12: Save Results to Drive

```
import shutil
from datetime import datetime

# Create timestamped results folder
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
results_dest = f'{DRIVE_ROOT}/Ego4d_STA/runs/Track_B_VideoMAE_{timestamp}'
os.makedirs(results_dest, exist_ok=True)

# Copy checkpoints
ckpt_src = f'{LOCAL_EXT}/runs/Track_B/checkpoints'
if os.path.exists(ckpt_src):
    shutil.copytree(ckpt_src, f'{results_dest}/checkpoints')
    print(f"✓ Saved checkpoints to {results_dest}/checkpoints")

# Copy run logs
for run_dir in glob.glob(f'{LOCAL_EXT}/runs/Track_B/trackB_*'):
    run_name = os.path.basename(run_dir)
    shutil.copytree(run_dir, f'{results_dest}/{run_name}')
    print(f"✓ Saved run: {run_name}")

print(f"\n✓ All results saved to: {results_dest}")
!ls -la "{results_dest}"
```

Cell 13 (Optional): Run ResNet18 Baseline for Comparison

```
# Run baseline with ResNet18 backbone for comparison
!python -m trackB.trackB_train_loader \
    --config trackB \
    --epochs 20 \
    --batch_size 8
```

Expected Training Metrics

During training, you'll see:

```
[TrackB main] Epoch 1/20: 100%|██████████| 187/187 [05:23<00:00]
    loss=0.6234, cls=0.5891, ttc=0.3421
[TrackB eval] Validation:
    mAP=0.2345, acc=0.6789, ttc_mae=0.4567
```

Key metrics:

Metric	Description	Target
loss	Total loss (cls + ttc)	↓ Lower is better
cls	Classification loss	↓ Lower is better
ttc	Time-to-contact loss	↓ Lower is better
mAP	Mean Average Precision	↑ Higher is better
acc	Binary accuracy	↑ Higher is better
ttc_mae	TTC Mean Absolute Error	↓ Lower is better

Expected Training Time

Config	GPU (Colab T4)	CPU
Demo (10 steps)	~1-2 min	~10 min
1 epoch	~10-15 min	~3 hours
20 epochs	~4-5 hours	~60+ hours

Troubleshooting

"No module named 'trackB'"

Make sure you're in the correct directory:

```
os.chdir(f'{REPO_PATH}/local_extraction')
```

"VideoMAE checkpoint not found"

Check the checkpoint path and copy it:

```
shutil.copy(VIDEOMAE_CKPT, local_ckpt)
```

"CUDA out of memory"

Reduce batch size:

```
!python -m trackB.trackB_train_loader --config trackB_videomae_ego --  
batch_size 4
```

"Frames not found"

Verify the symlink is correct:

```
print(os.listdir(f'{LOCAL_EXT}/v2/extracted_frames')[:5])
```

Summary Checklist

Before running training, verify:

- ☐ GPU is available (`nvidia-smi` shows GPU)
- ☐ Drive is mounted
- ☐ VideoMAE checkpoint copied to local
- ☐ Frames symlinked correctly
- ☐ Manifests copied (head_train.jsonl, head_val.jsonl)
- ☐ Demo runs successfully
- ☐ Config loads correctly (video_backbone=videomae_ego)

Final Drive Structure

After training, your Drive should have:

```
Ego4d_STA/
├── videomae/
│   └── checkpoints/
│       └── videomae_ego_scratch_last.pt    # Input checkpoint
├── runs/
│   ├── Track_A/
│   │   ├── trackA_stageB_YYYYMMDD_HHMMSS/ # Input manifests
│   │   │   ├── head_train.jsonl
│   │   │   └── head_val.jsonl
│   │   └── Track_B_VideoMAE_YYYYMMDD_HHMMSS/ # VideoMAE results
│   │       ├── checkpoints/
│   │       │   └── trackB_best.pt
│   │       ├── trackB_YYYYMMDD_HHMMSS/
│   │       │   ├── run_log.json
│   │       │   └── metrics/
│   └── Track_B_ResNet18_YYYYMMDD_HHMMSS/    # ResNet18 results
│       ├── checkpoints/
│       │   └── trackB_best.pt
│       ├── trackB_YYYYMMDD_HHMMSS/
│       │   ├── run_log.json
│       │   └── metrics/
```

Part 2: ResNet18 Baseline Training on Colab

This section explains how to run the ResNet18 baseline for comparison with VideoMAE.

Data Required for ResNet18

ResNet18 is **simpler** - it doesn't need the VideoMAE checkpoint:

Data	Required	Path
Extracted Frames	☒ Yes	ego4d_data/v2/extracted_frames/
Train Manifest	☒ Yes	Ego4d_STA/runs/Track_A/trackA_stageB_*/head_train.jsonl
Val Manifest	☒ Yes	Ego4d_STA/runs/Track_A/trackA_stageB_*/head_val.jsonl
VideoMAE Checkpoint	☒ No	Not needed

ResNet18 Colab Notebook

Cell 1: Check GPU

```
!nvidia-smi
import torch
print(f"CUDA available: {torch.cuda.is_available()}")
```

Cell 2: Mount Google Drive

```
from google.colab import drive
drive.mount('/content/drive')
```

Cell 3: Install Dependencies

```
!pip install -q torch torchvision tqdm pillow pyyaml
# Note: transformers NOT needed for ResNet18
```

Cell 4: Clone Repository

```
import os

REPO_PATH = '/content/Ego4d-LiteSTA'

if not os.path.exists(REPO_PATH):
    !git clone https://github.com/saeedzns/Ego4d-LiteSTA.git {REPO_PATH}
else:
    !cd {REPO_PATH} && git pull
```

Cell 5: Configure Paths (ResNet18)

```
import os
import sys

# === EDIT THESE PATHS TO MATCH YOUR DRIVE ===
DRIVE_ROOT = '/content/drive/MyDrive'
```

```
# Extracted frames directory
FRAMES_ROOT = f'{DRIVE_ROOT}/ego4d_data/v2/extracted_frames'

# Track A manifests directory
MANIFESTS_ROOT = f'{DRIVE_ROOT}/Ego4d_STA/runs/Track_A'

# Verify paths
print("Checking paths...")
print(f"✓ Frames root: {os.path.exists(FRAMES_ROOT)}")
print(f"✓ Manifests root: {os.path.exists(MANIFESTS_ROOT)}")

# Note: No VideoMAE checkpoint needed for ResNet18!
```

Cell 6: Setup Local Directory Structure (ResNet18)

```
import shutil
import glob

LOCAL_EXT = f'{REPO_PATH}/local_extraction'

# Create directories
os.makedirs(f'{LOCAL_EXT}/v2', exist_ok=True)
os.makedirs(f'{LOCAL_EXT}/runs/Track_A', exist_ok=True)

# 1. Symlink frames (don't copy - too large)
local_frames = f'{LOCAL_EXT}/v2/extracted_frames'
if not os.path.exists(local_frames):
    os.symlink(FRAMES_ROOT, local_frames)
    print(f"✓ Linked frames: {local_frames}")

# 2. Copy manifests (find latest stageB run)
stageb_runs = sorted(glob.glob(f'{MANIFESTS_ROOT}/trackA_stageB_*'))
if stageb_runs:
    latest_run = stageb_runs[-1]
    run_name = os.path.basename(latest_run)
    local_run = f'{LOCAL_EXT}/runs/Track_A/{run_name}'

    if not os.path.exists(local_run):
        shutil.copytree(latest_run, local_run)
        print(f"✓ Copied manifests: {local_run}")

# Verify manifest files
train_manifest = f'{local_run}/head_train.jsonl'
val_manifest = f'{local_run}/head_val.jsonl'
print(f"✓ Train manifest exists: {os.path.exists(train_manifest)}")
print(f"✓ Val manifest exists: {os.path.exists(val_manifest)}")
else:
    print("✗ No Track A Stage B runs found!")
```

Cell 7: Set Working Directory

```
os.chdir(LOCAL_EXT)
sys.path.insert(0, LOCAL_EXT)
print(f"Working directory: {os.getcwd()}")
```

Cell 8: Test ResNet18 Config Loading

```
from core import load_config

cfg = load_config('trackB', colab=True)

print("=== ResNet18 Config ===")
print(f"video_backbone: {cfg.get('model.tokenizer.video_backbone')}") #
Should be 'resnet18'
print(f"time_len: {cfg.get('model.tokenizer.time_len')}") #
Should be 8
print(f"epochs: {cfg.get('training.epochs')}")
print(f"batch_size: {cfg.get('training.batch_size')}")
print(f"lr: {cfg.get('training.lr')}")
print(f"projector_in_dim: {cfg.get('model.projector.in_dim')}") #
Should be 512
```

Cell 9: Run Demo (Quick Verification)

```
# Run 10-step demo to verify everything works
!python -m trackB.trackB_train_loader --config trackB --demo
```

Expected output:

```
[TrackB] Loading config: trackB
[TrackB] Video backbone: resnet18
[TrackB] Projector: 512 -> 256
[TrackB demo] steps (loader): 100%|██████| 10/10 [00:XX<00:00]
```

Cell 10: Full ResNet18 Training

```
# Full training with ResNet18 backbone
# ResNet18 is MUCH faster than VideoMAE

!python -m trackB.trackB_train_loader \
    --config trackB \
    --epochs 20 \
    --batch_size 8
```

Alternative: Quick test with fewer epochs:

```
!python -m trackB.trackB_train_loader \
    --config trackB \
    --epochs 3 \
    --batch_size 8
```

Cell 11: Save ResNet18 Results to Drive

```
import shutil
from datetime import datetime

# Create timestamped results folder
timestamp = datetime.now().strftime('%Y%m%d_%H%M%S')
results_dest = f'{DRIVE_ROOT}/Ego4d_STA/runs/Track_B_ResNet18_{timestamp}'
os.makedirs(results_dest, exist_ok=True)

# Copy checkpoints
ckpt_src = f'{LOCAL_EXT}/runs/Track_B/checkpoints'
if os.path.exists(ckpt_src):
    shutil.copytree(ckpt_src, f'{results_dest}/checkpoints')
    print(f"✓ Saved checkpoints to {results_dest}/checkpoints")

# Copy run logs
for run_dir in glob.glob(f'{LOCAL_EXT}/runs/Track_B/trackB_*'):
    run_name = os.path.basename(run_dir)
    shutil.copytree(run_dir, f'{results_dest}/{run_name}')
    print(f"✓ Saved run: {run_name}")

print(f"\n✓ All results saved to: {results_dest}")
!ls -la "{results_dest}"
```

Expected Training Time Comparison

Model	GPU (Colab T4)	CPU
ResNet18 Demo (10 steps)	~30 sec	~1 min
ResNet18 1 epoch	~3-5 min	~15 min
ResNet18 20 epochs	~1-2 hours	~5 hours
VideoMAE Demo (10 steps)	~1-2 min	~10 min
VideoMAE 1 epoch	~10-15 min	~3 hours
VideoMAE 20 epochs	~4-5 hours	~60+ hours

Key Differences: ResNet18 vs VideoMAE

Aspect	ResNet18	VideoMAE
Backbone type	2D CNN (per-frame)	3D Transformer (video)
Token dimensions	(49, 512) = 7×7 grid	(1568, 768) = 8×14×14
Frames used	8 frames	16 frames
Pretrained on	ImageNet (web images)	Ego4D (egocentric video)
Speed	~5× faster	Slower but richer features
Dependencies	torchvision only	+ transformers

Running Both for Comparison

To compare VideoMAE vs ResNet18, run both in the same Colab session:

```
# 1. Run ResNet18 baseline first (faster)
!python -m trackB.trackB_train_loader --config trackB --epochs 20

# Save ResNet18 results
!cp -r runs/Track_B /content/drive/MyDrive/Ego4d_STA/runs/Track_B_ResNet18/

# Clear runs folder
!rm -rf runs/Track_B/*

# 2. Run VideoMAE
!python -m trackB.trackB_train_loader --config trackB_videomae_ego --epochs 20

# Save VideoMAE results
!cp -r runs/Track_B /content/drive/MyDrive/Ego4d_STA/runs/Track_B_VideoMAE/
```

ResNet18 Checklist

Before running ResNet18 training, verify:

- ☐ GPU is available (`nvidia-smi` shows GPU)
- ☐ Drive is mounted
- ☐ Frames symlinked correctly
- ☐ Manifests copied (head_train.jsonl, head_val.jsonl)
- ☐ Demo runs successfully
- ☐ Config shows `video_backbone=resnet18`
- ☐ Config shows `projector_in_dim=512`